

Guancheng Rong
1856981

1.

We could define,
when $n \leq 5$
 $T(n) = O(1)$
When $n > 5$,
 $T(n) = T(4n/5) + O(1)$,

when $n \leq 5$
Times >>> RemainNum
1 >>> n-1
2 >>> n-2
3 >>> n-3
...
5 >>> 0
Therefore, when $n \leq 5$, $O(1)$

When $n > 5$,
1/5 - search
Times >>> RemainNum
1 >>> $n - n // 5$ >>> $\frac{4}{5}n$
2 >>> $(n - n // 5) - (n - n // 5) // 5$ >>> $(\frac{4}{5})(\frac{4}{5})n$
3 >>> $(\frac{4}{5})(\frac{4}{5})n - (\frac{4}{5})(\frac{4}{5})n // 5$ >>> $(\frac{4}{5})(\frac{4}{5})(\frac{4}{5})n$
...
...
...
k-1 >>> $[(\frac{4}{5})^{k-1}]n$
k >>> $[(\frac{4}{5})^k]n$

because all comparison process only one time — $O(1)$,
so, when $n > 5$, $k = \log_{1.25}(n)$,
 $k \cdot O(1) = O(\log n)$

In summary,
when $n \leq 5$, $O(1)$
when $n > 5$, $O(\log n)$

2.

(a).

When $0 < n \leq 1/a$,

So, $n/(1/a) < 1$

Therefore, we can't use the $1/a$ -search algorithm,
we can only use the brute-force algorithm.

So

Times >>> RemainNum

1 >>> $1/a - 1$

2 >>> $1/a - 2$

...

...

$k-1$ >>> $1/a - (k-1) = 1/a - k + 1$

so, $O(1/a)$

(b).

We can define,

When $n > 1/a$,

$T(n) = T((1-a)n) + O(1)$;

when $n \leq (1/a)$,

$T(n) = O(1/a)$

When $n > 1/a$,

Times >>> RemainNum

1 >>> $n - n // (1/a) >>> n(1 - a)$

2 >>> $n(1 - a) - n(1-a) // (1/a) >>> n(1 - a)(1 - a)$

...

...

...

$k-1$ >>> $n(1-a)(1-a) \dots (1-a) >>> n(1-a)^{(k-1)}$

k >>> $n(1-a)(1-a) \dots (1-a) >>> n(1-a)^k$

so $O(\log_{1/1-a}(n))$

$$O(1/a + \log_{1/1-a}(n)) = O(1/a) + O(\log_{1/1-a}(n))$$

If we want to find any number in one n length of array with a -search algorithms.

We will first recurse the array with a -search algorithm until $(1-a)^{k1} * n \leq 1/a$. Then,
we have to use the brute force algorithm to search the array one by one, which will use the $k2$ times at the worst situation.

In the most complex situation,

$k1$ >>>> $O(\log_{1/1-a}(n))$

$k_2 \gg \gg \gg O(1/a)$
 $K \gg \gg \gg k_1 + k_2 \gg \gg \gg O(1/a + \log_{1/1-a}(n))$

Above all, the a-search running complexity must be $O(1/a + \log_{1/1-a}(n))$

3.

(a).

we assume one array $A[a_1, a_2, a_3, \dots, a_{n-1}, a_n]$

Algorithm:

```

for i = 0 to n-1:
    for j = i to n-1:
        if A[i] + A[j] == k:
            return true
return false

```

When we use the brute-force algorithms, we could set a_1 as a benchmark, plus each number in the array — $A[a_1, a_2, a_3, \dots, a_{n-1}, a_n]$. Then we set a_2 as a benchmark, plus each number in the array except a_1 until a_{n-1} becomes a benchmark, it will plus with a_n .

The whole progress is :

$$n + (n - 1) + \dots + 3 + 2 + 1 = (n+1) * n / 2$$

So, $O(n^2)$

(b)

$A[a_1, a_2, a_3, \dots, a_{n-1}, a_n]$

Algorithm:

we could start from a_1 . we need to find $k - a_1$ in the remaining array. Therefore, we could use binary-search in array $A[a_1, a_2, \dots, a_{n-1}, a_n]$ to save time. Then, we start from a_2 , and use binary search to find the target — $k - a_2$ in the remaining array $A[a_2, a_3, \dots, a_{n-1}, a_n]$. This process is repeated until the sum equals $k - a_m$.

Proof:

```

Loop1 >>> Loop2
1 >>> k - a1
        binary search in A[ a1, a2, ..., an-1, an] for k - a1 value
        O( log n )
2 >>> k - a2
        binary search in A[ a2, a3, ..., an-1, an] for k - a2 value
        O( log (n-1) )
3 >>> k - a3
        binary search in A[ a3, a4, ..., an-1, an] for k - a3 value
        O( log (n-2) )
...
...
...
n-2 >>> k - an-2
        binary search in A[ an-2, an-1, an] for k - an-2 value
        O( log (3) )
n-1 >>> k - an-1
        binary search in A[ an-1, an] for k - an-1 value
        O( log (2) )

```

$$O(\log n) + O(\log (n-1)) + O(\log (n-2)) + \dots + O(\log (3)) + O(\log (2)) \\ \leq O(\log n) + O(\log n) + \dots + O(\log n) = O(n \log n)$$

Therefore, the complexity of running time is $O(n \log n)$

(c)

$A[a_1, a_2, a_3, \dots, a_{n-1}, a_n]$

Algorithm:

We choose the start and the end of the array — a_1, a_n . Then we add them with each other and compare the result with k value. If the result is smaller than k , we can choose the larger value from the start — a_2 ; if the result is larger than k , we would choose the smaller value from the end — a_{n-1} . This process is repeated until the sum equals k . Or until the two sides meet with each other, there is no matching value.

```

i = 1
j = n
while i ≤ j:
    sum = A[i] + A[j]
    if sum == k:
        return true
    else if sum < k:
        i = i + 1
    else:
        j = j - 1
return false

```

Prove for the correctness:

We could assume there is a pair of accessible value $A[i^*] + A[j^*] = k$. Therefore, we could get $i \leq i^*$ & $j^* \leq j$.

situation.1

When $A[i] + A[j] = k$,

The algorithm is right.

situation.2

When $A[i] + A[j] < k$,

we could assume $i = i^*$,

$A[i] + A[j] < k = A[i^*] + A[j^*] = A[i] + A[j^*]$

so, $A[j] < A[j^*] \implies j < j^*$.

However, it's impossible, because $j^* \leq j$.

so $i < i^*$, then $i+1 \leq i^*$

situation.3

When $A[i] + A[j] > k$,

This is similar to the situation.2 proof.

If the algorithm does not return "yes" midway, the loop continues. By the invariant, $i \leq i^* \leq j^* \leq j$, therefore $i \leq j$ always holds, and the loop will continue until an iteration hits $A[i] + A[j] = k$. However, if a valid solution exists, the algorithm will reach the state of $i = i^*$ and $j = j^*$ at some step (guaranteed by the tightening process of the invariant), at which point it returns "true".

In summary, the algorithm is correct.

Therefore, when we start from both ends and gradually increase and decrease, the worst situation is the two values will meet with each other at one pair of neighbour numbers in the array. So, the complexity of running time is $O(n)$.